

אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Hatchala - Introduction to computer science in Java - Fall 26'-27'

Department of Computer Science and Information

Ben-Gurion University of the Negev

Based on both Dr. Assaf Zaritsky slides for "Intro to CS" (IS)
and slides for "Introduction to CS" (CS) courses at BGU

Lecture 3: Functions

Last lesson

- Conditional statements
- loops: while / for loops, Java basic Arrays

Plan for today

- Functions
  - **What are they good for**
  - Built-in functions
  - Defining new function

- Good code practice
- Tests
- Tools & Tips

Code duplications are bad

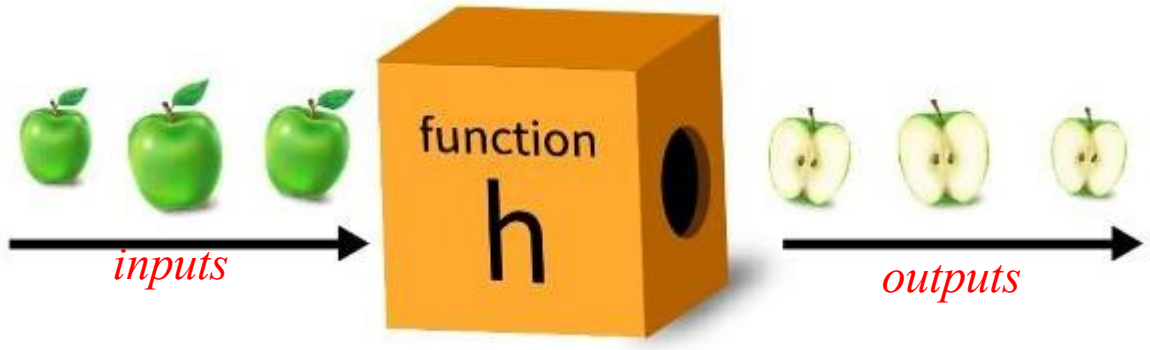
- Duplicated bugs (harder to fix later)
- Longer programs
- Copy & paste errors

Rule of Thumb - write the code once!

- Code reuse: use the same code several times with different arguments

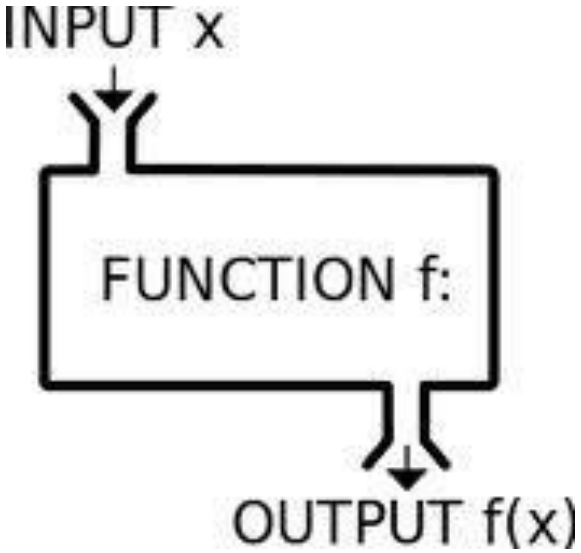
What is a **function** in programming ?

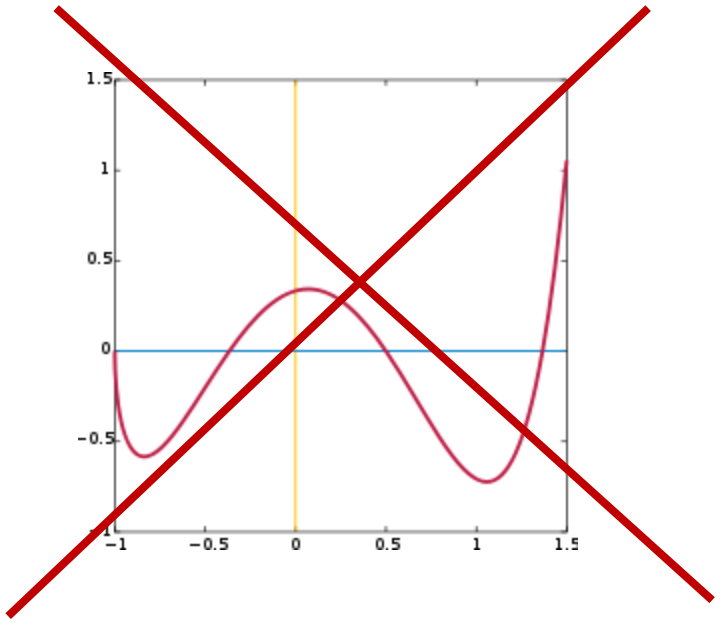
- A function is a block of organized, reusable code that is used to perform a single, well-defined action
- Function's *inputs* determines the *outputs*
- Specific input is called an *argument* of a function
- Functions provide better modularity for your application and a high degree of code reusing



Function – definition II

A named sequence of statements that performs a specific task independently of the remaining code





Modularity



Ernie wrote code calculating a^b.

To use the code in 8 different calculations, he:

- copy & pasted
- assigned arguments

Eight times

Bert came up with a more efficient algorithm to calculate a^b

To update Ernie's code, he went over the eight calculations ☹

Modularity enables code reuse!

"**Modularity** is the degree to which a system's components may be separated and recombined" (Wikipedia)

- Top-down design
- Improves maintainability
- Enforces logical boundaries between components



Abstraction

Calculate $\ln\left(\sqrt{a^2 + b^2}\right)$

ln of **sqrt** of **sum** of **squares**

Five calls to four different functions!

→Abstraction

→Readability

Abstraction

Calculate $\ln\left(\sqrt{a^2 + b^2}\right)$

ln of **sqrt** of **sum** of **squares**

```
double result = Math.log(Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2)));
```

`Math.pow(a, 2)` → squares a

`Math.pow(b, 2)` → squares b

`Math.sqrt(...)` → takes the square root of their sum

`Math.log(...)` → computes the natural logarithm (ln)

So – why use functions?

- Modularity - Break a task into smaller sub-tasks (divide and conquer), enables code reuse
  - Maintenance - Solve bugs **once**
- Abstraction – "Hiding" complexity
- Readability – Main code is shorter, implementation details are hidden within functions

Plan for today

- Functions
  - What are they good for
  - **Built-in Functions**
  - Defining New Functions

Built-in functions

- We are using functions already
  - System.out.println("Hello world!") פונקציית מערכת
  - myScanner.nextInt() שיטה של אובייקט
  - Math.random() פונקציית מחלקה
  - main(String[] args) הפונקציה הראשית

Plan for today

- Functions
  - What are they good for
  - Built-in functions
  - **Defining new functions**

Functions- structure

Defining a function:

```
public static <return value type> <functionName> (<arg1 type>, <arg1 name>,
                                                   <arg2 type>, <arg2 name>,...){
        statement1
        statement2
        …
        return <some var or value>;
    }
        # return is optional if return type is void
```

Calling a function:

```
var1 = functionName(val1, val2,...)
```

Function input / output

Input: arguments

Any type: int / float / boolean / array

Output: the return command

Returns a value to the function's caller

- returned value – any Java type
- return stops immediately the function's execution and returns to the caller!
- (return is different from println!)

Good coding practices

Good coding practices

- Divide & conquer:
  - Disassemble into smaller sub-problems.
  - Solve each sub-problem independently.
- Documentation
  - Meaningful functions' and variables' names.
  - Arguments & output description for each function.
  - Summarize each step